

System Verification and Validation Plan for Software Engineering

Team 11, technically functional

Matthew Huynh

Cieran Diebolt

Vaisnavi Shanthamoorthy

Maham Siddiqui

Eman Ashraf

October 27, 2025

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to “fake it”, or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1	Symbols, Abbreviations, and Acronyms	iv
2	General Information	1
2.1	Summary	1
2.2	Objectives	1
2.3	Extras	2
2.4	Relevant Documentation	3
3	Plan	3
3.1	Verification and Validation Team	3
3.2	SRS Verification	3
3.3	Design Verification	5
3.4	Verification and Validation Plan Verification	5
3.5	Implementation Verification	5
3.6	Automated Testing and Verification Tools	6
3.7	Software Validation	6
4	System Tests	6
4.1	Tests for Functional Requirements	7
4.1.1	Area of Testing1	7
4.1.2	Area of Testing2	8
4.2	Tests for Nonfunctional Requirements	8
4.2.1	Area of Testing1	8
4.2.2	Area of Testing2	9
4.3	Traceability Between Test Cases and Requirements	9
5	Unit Test Description	9
5.1	Unit Testing Scope	10
5.2	Tests for Functional Requirements	10
5.2.1	Module 1	10
5.2.2	Module 2	11
5.3	Tests for Nonfunctional Requirements	11
5.3.1	Module ?	11
5.3.2	Module ?	12
5.4	Traceability Between Test Cases and Modules	12

6	Appendix	14
6.1	Symbolic Parameters	14
6.2	Usability Survey Questions?	14

List of Tables

1	Verification and Validation Team	4
---	--	---

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

Refer to Section 4 in the [SRS document](#) for a table of many more symbols, abbreviations, acronyms, and definitions.

This document aims to outline the verification and validation that the Physiotherapy Companion app will go through during development. The document begins with a brief description as to what the software is, what it is meant to achieve, and some relevant information before explaining in detail the standards and metrics that will be used to evaluate the system.

2 General Information

2.1 Summary

The software focused on in this document is the Physiotherapy Companion application, built for smart devices with camera access to assess a patient undergoing physiotherapy on their form during assigned physiotherapy exercises.

The application will provide support and guidance for the exercise and make real-time suggestions to improve the user's form using a video-feed. It will also provide overviews of previous exercises to allow users to see progress and consistency, assisting in forming healthy habits towards recovery and reinforce confidence.

The application will also support a more administrative function for physiotherapists, allowing them to view exercise summaries for patients assigned to them. Additionally, physiotherapists may leave comments on summaries to ensure consistent and timely usage of the application for healthy exercising.

2.2 Objectives

The objectives of the testing expanded upon within this document focus on general system usability, the security of user profiles and video-feed data, and the verification of stakeholder requirements.

Usage of the system should be easily explainable for first-time users with simple dialogue popups and limited visible guiding from one function to the next. Additionally, for the video-feed portion of the application's usage, the user should be reasonably guided through the exercise and process to set them up for successful exercise execution and form recommendation acceptance.

User profile and video-feed data should be securely stored, whether locally or within a remote database accessed by the system. User data security has

become an increasingly sensitive topic within the software industry of late and users may question how their data is being stored and utilized. Users should be aware of the usage of their data and the system should ensure unauthorized access outside of the communicated plan is never possible.

Meeting stakeholder requirements, whether they be data restrictive or process flow in nature, is what will validate the core features of the system. If users do not find value in the expected usage expected from the system, then, ultimately, the system has failed to perform its primary goal. Ensuring stakeholder requirements and feedback are incorporated into the system as it is developed, both initially and onward, will ensure the system is both wanted and usable to the intended audience.

Some out-of-scope objectives include performance and user-interface quality. In this sense, quality means the overall refinement and how polished the interface looks. The implementation should not inhibit usability, but the design need not be too meticulous and visually pleasing.

For performance, the system should respond in a timely manner that does not frustrate a user's ability to perform the main activities of the system. Designing an optimized system which may use minimal amounts of resources is not one of the current objectives of the system. These things could both be expanded into and developed, but for now are considered out of scope.

2.3 Extras

The extras set out for the system consist of a Design Thinking Document and a User Instructional Video.

The Design Thinking Document aims to explain the process behind, and work done towards, the final design of the system. This document explains the overall flow of processing through the system and the interfacing elements, external libraries utilized, and more. The document flow is iterative, as is the design process, and aims to communicate the entire design process of the system.

The User Instructional Video aims to communicate the method of usage of the system to the primary stakeholders. The video will demonstrate the usage of the dashboard and analytics display, the video-feed form recommendation function, as well as an explanation of the overall feedback provided for each completed exercise. The instructions will be patient-centred. For the physiotherapist view the patient list, comment features, and analytical display function will be explained.

2.4 Relevant Documentation

- Problem Statement and Goals: [Ashraf et al. \(2025d\)](#)
- Development Plan: [Ashraf et al. \(2025b\)](#)
- System Requirements Specification: [Ashraf et al. \(2025e\)](#)
- Hazard Analysis: [Ashraf et al. \(2025c\)](#)
- User Instructional Video: [Ashraf et al. \(2025f\)](#)
- Design Thinking Document: [Ashraf et al. \(2025a\)](#)

All of the above document are important artifacts for the implementation of the system. These documents outline how the implementation will be approached and what the implementation will follow.

More documents to come here for external libraries such as OpenCV, pytest, Python Extension, and more when it is determined which libraries and external softwares will be used.

3 Plan

This section will focus on the methods to verify and validate artifacts of the project. The methods for each facet of the system will be listed below, and additional information including possible questionnaire questions can be found in the appendix at the end of this document. The section will start primarily with verification and move to validation at the end.

3.1 Verification and Validation Team

See *Table 1: Verification and Validation Team*.

3.2 SRS Verification

To verify the SRS, several approaches will be used across different sections of the document. In general, the document will receive verification through feedback from 4G06 teaching assistants, peer-review feedback from another design team, and a check in with a project advisor to ensure the document

Role	Member(s)	Description
Test Development	Maham, Eman	Development of tests for the system and system units based on functional and non-functional requirements listed within the SRS document.
Test Implementation	Cieran	Implementation of a test suite for feasibly software testable requirements, developed based on the tests made by the test development team. Also assurance of software coverage by the suite.
Test Execution	Matthew	Running and verification of the test suite.
Requirements Verification	Vaisnavi	Verification and tracking of requirements to ensure each has been met and tested effectively.
Stakeholder Validation	All	Validation with stakeholders and external users to ensure the system developed performs as users expect; focus on usability.

Table 1: Verification and Validation Team

represents the correct motivation for the project. This check-in will be a self review by the advisor followed by a scheduled meeting to present any probable modifications to the requirements. Based on feedback from the advisor, issues will be created and handled.

To verify the functional requirements within the SRS, the above will be used in addition to software checks for functionality and a requirements checklist which will be updated upon checking functional requirements within the implementation. For verification of non-functional requirements within the SRS, in addition to the general checks, there will be a questionnaire of the software's usage and another checklist with a more flexible scaling system for the NFRs.

3.3 Design Verification

Verification of the design will rely on feedback from 4G06 teaching assistants, peer-review feedback provided by colleagues on another design team, as well as advisor and stakeholder inquiries. The feedback from teaching assistants and peer-review feedback will be created as issues within our repository and handled accordingly. For advisor and stakeholder inquiry, there will be software handouts along with a questionnaire which is expanded on more at the end of this section.

3.4 Verification and Validation Plan Verification

This document will also be reviewed by the teaching assistants and provided peer-review by another design team, creating issues within the GitHub repository to be dealt with accordingly.

In addition, the test suite derived from this document will be verified via code coverage and mutation testing on the software. For validation, the final outcome of each test will be discussed in the meeting with the project advisor to ensure the system is doing what it should in each test case.

3.5 Implementation Verification

For testing the implemented system, there will be a software testing suite developed based on the tests listed in sections 4 and 5 of this document. Additional testing by the team includes code walk-throughs of critical sections

both during development and upon reaching milestones within the development cycle. There will also be static analysis on each pull request to GitHub, ensuring naming conventions and best practice are followed throughout the project.

3.6 Automated Testing and Verification Tools

Automated testing includes the testing suite previously mentioned in this section; derived from the tests found in this document which are based upon the requirements found within the [SRS document](#) of this project. These tests will be verified themselves with mutation testing and code coverage assessments. A testing suite such as pytest or Python Extension will be utilized to implement the tests mentioned in further sections of this document. These suits contain their own internal tools for checking code coverage, and the GitHub will be using continuous integration to ensure base cases continue to pass through all pull requests as well as general compilation checks.

During development, pylint will ensure the coding standard is adhered to and continuous development within GitHub will ensure no slip-ups on merging.

3.7 Software Validation

For software validation, to ensure the correct system was designed to solve the problems outlined within the problem statement document, there will be system walkthroughs and interviews with stakeholders and the project advisor. These interviews will assess time to use the systems functions, understanding of the system and how it processes data, as well as a general acceptance of the system. The interview will end with questions outlined in the appendix of this document. (Section 6.2)

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

Control: Manual versus Automatic

Initial State:

Input:

Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

Control: Manual versus Automatic

Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

Type: Functional, Dynamic, Manual, Static etc.

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

Type: Functional, Dynamic, Manual, Static etc.

Initial State:

Input:

Output:

How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

Please refer to the following Microsoft Excel file: [Traceability Matrix](#)

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, your code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

Type: Functional, Dynamic, Manual, Static etc.

Initial State:

Input:

Output:

How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all of the modules have been considered. —SS]

References

- Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Design thinking document. <https://github.com/M9Huynh/technically-functional/tree/32182cfc56c602466e270b7fcbad01d892911ea1/docs/Extras/DesignThinking>, 2025a.
- Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Development plan. <https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/DevelopmentPlan/DevelopmentPlan.pdf>, 2025b.
- Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Hazard analysis. <https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/HazardAnalysis/HazardAnalysis.pdf>, 2025c.
- Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Problem statement and goals. <https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/ProblemStatementAndGoals/ProblemStatement.pdf>, 2025d.
- Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. System requirements specification. <https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/SRS/SRS.pdf>, 2025e.
- Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. User instructional video. <https://github.com/M9Huynh/technically-functional/tree/32182cfc56c602466e270b7fcbad01d892911ea1/docs/Extras/UserInstructionalVideo>, 2025f.

6 Appendix

Additional project information to expand on the previous sections. Mostly example or sample information at this time.

6.1 Symbolic Parameters

Example script layout for system testing:

```
/*  
# Create user account testing  
New account creation (user_acc_create)  
Invalid format of field inputs (invalid_formatCreate)  
Invalid sign up credentials (cannot_verify_creds)  
Presence of an existing account (acc_exists)  
  
# PT focused testing  
PT successfully sends invite to patient (PT_sends_invite)  
PT adds/modified patient exercise (PT_adjusts_exercise)  
PT deletes patient account (PT_deletes_patient_acc)  
  
# Patient focused testing  
System_exercise_overview  
Video_store  
Video_retrieval  
System_analysis_feedback  
Post_ex_patient_feedback  
Incorrect_form_adjustment  
  
# NFR testing  
Ex_int_speed  
Fail_recovery  
Interrupted_vid  
/
```

6.2 Usability Survey Questions?

Advisor SRS Questions:

1. Are there any sections of the requirements which are found to be confusing or hard to measure?
2. Are there any sections of the system you feel the requirements do not expand on?
3. Are there any sections of the system you feel the requirements expand on that is out of scope?
4. Do you feel the requirements are extensive enough?
5. Do you feel the requirements are tracked well?

Design Questionnaire Questions:

“Strongly Disagree” to “Strongly Agree” style questioning.

1. The system navigation is intuitive.
2. The system’s functions are all easily accessible.
3. The system helped set up your exercises for success.
4. The system enables you to complete your exercises with better guidance.
5. The system enables you to complete your exercises with better confidence.
6. The feedback returned during your exercise was helpful.
7. The overview data from your exercise was helpful.
8. You would you use this software for your exercises again.

Could have a final open-ended question based on user pain points with the system I.e. what did not work well for individuals.

Software Validation Questionnaire Questions:

“Strongly Disagree” to “Strongly Agree” style questioning.

1. This software assisted you in completing your exercises.
2. This software was easy to use.

3. This software was easy to navigate.
4. The feedback from the system assisted you in fixing your form.
5. The feedback from the system generated greater confidence in your form.
6. The feedback from the system was meaningful and helpful.
7. The data generated from your exercises was helpful.
8. The system responded how you expected it to during usage.
9. The system clearly described what was going on.
10. You felt adequately trained to use the system from the system dialogues.
11. You felt safe while using the system.

Ui_testing Survey:

1. How would you rate your overall experience with the application (1 - poor, 5 - excellent)?
2. How would you rate the visual appeal of the application (1 - poor, 5 - excellent)?
3. How easy was it to navigate through the application? (1 - difficult, 5 - easy)
 - (a) Were the tutorials helpful? (1 - not helpful, 5 - very helpful)
 - (b) How logical was the arrangement of the different components within the system (1 - very illogical, 5 - very logical)?
4. Which areas could use some improvement?
 - (a) Navigation
 - (b) Menu structure
 - (c) Colour/contrast scheme
 - (d) Error messages

- (e) Other (With open text entry)

Patient Post-Exercise Feedback Form:

1. How did you feel before the exercise? (1 - good, 5-bad)
 - (a) Did you have tightness? Yes/no
 - (b) Did you have any pain? Yes/no.
 - i. How bad was your pain? (1 - no pain, 10 - unbearable pain)
 - ii. What was the perceived level of pain/difficulty? (1 - very easy, 10 - very difficult)
 - iii. Where was your pain located? (Open text entry)
 - (c) Have you noticed any recent changes? (Open text entry)
2. If you had any pain DURING the exercise, how bad was it? (1 - no pain, 10 - unbearable pain)
 - (a) Did you have tightness? Yes/no
 - (b) Did you have any pain? Yes/no.
 - i. How bad was your pain? (1 - no pain, 10 - unbearable pain)
 - ii. What was the perceived level of pain/difficulty? (1 - very easy, 10 - very difficult)
 - iii. Where was your pain located? (Open text entry)
 - (c) Have you noticed any recent changes? (Open text entry)
4. How challenging was the exercise today? (1 - easy, 5 - very hard)
 - (a) Do you have any feedback for your physiotherapist? (Open text entry)
5. In your opinion, is this treatment plan helping your health? Yes/no. Choose the option that describes how the plan is helping (Can choose multiple).
 - (a) My movement
 - (b) My flexibility

(c) My pain

(d) My mood

6. In your opinion, how difficult is the current prescribed exercise plan?
(1 - easy, 5 - very difficult)

(a) Should your physiotherapist change the difficulty of your plan? (1
- no change, 5 - Large increase in difficulty)

Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Vaisnavi - Reflection

1. I think one main thing that went well while writing this deliverable was our communication as a group and how that played into the division

of work. We had two sub-teams where Matthew and I were focusing on the POC, while Cieran, Maham and Eman were focused on this VnV deliverable. Our communication throughout aided in making this process of working on the deliverable seamless as everyone seemed to be very looped in on what each sub-team was working on. We had our weekly meetings to touch base and ask for any help if it was needed, and it worked very well.

2. I think one of the main pain points we came across was as a result of us working on the POC, it was a little bit hard at the start to fully keep up with updates for the VnV Plan. However, as mentioned above, with sharing our progress and updates during our team meetings, this helped in providing more clarity and consistency between all group members, and it allowed everyone to be on the same page.

3. **GROUP Q**

4. To improve the knowledge of OpenCV and MediaPipe pose detection for accurate testing of body landmark tracing, we could do the two approaches:
 - (a) Follow documentation and existing examples to understand how to properly utilize OpenCV and Mediapipe to accurately detect the varying body landmarks through pose detection.
 - (b) Try to work with small test cases and work on small files to test it out on different body landmarks and see how it works.

Maham - Reflection

1. The team was split into two groups: one was responsible for the POC demonstration, and the other was completing the VnV report. All members were in constant communication even though they had other (very) important deadlines to meet. The sub-teams functioned very well, because each team had members who were (somewhat) more comfortable in the area they were responsible for, and others that wanted to expand their skillset. There was a good balance present. In the end, all members delivered work of the highest calibre and I am happy to be a part of such a great team.

2. I do not think it was a pain point but I had underestimated the amount of thought and importance that my portion would entail. I think it was challenging balancing the high priority of this deliverable (my part) and my other important (health) engagements as well. However, I used all the time management skills that I have learned and effectively managed to chunk my work into more realizable chunks while ensuring I am taking care of my well-being and delivered some good quality work.

3. GROUP Q

4. I had previously acquired some knowledge on software testing and have always been fascinated by it. But to really master the application, I had to be very detail oriented. I did a deep dive into the various components of our system. However, I realized that I had to be careful and not make my focus too narrow - because I was developing system tests. I had to look at the overall picture. I did this in 2 ways. Firstly, I researched 'system testing' to further increase my knowledge and get a more concrete idea. However, since I had prior knowledge, I found most of the resources online just broke down system testing into different unit tests. I did not want to pursue that because Eman was working on the unit testing component. I spoke to my teammates on the possible overlap between unit and system testing, and was more confident in my (potential) analysis and approach. In the end, I refined my testing knowledge and by designing different test cases for any possible scenario, and making sure the system was approached from various aspects without being too focused on individual modules.

Matthew - Reflection

1. I think one thing that went well when writing this deliverable was how we divided these two weeks up. We had taken the advice from the professor to begin development on the POC while working on the VnV plan. The team was divided into myself and Vais on the POC while Cieran, Maham and Eman were working on the VnV plan. This allowed us to do brief check-ins with each other while beginning development and thus more time to develop before the POC demo.
2. I think some pain points during this deliverable was tied to external workloads. Since half of the deliverable encompassed reading week, this

meant that the team had midterms and assignments for other classes due before this deliverable and managing that was more difficult. We resolved this by being transparent with each of our workloads, by letting each other know in our Teams chat if we were going to be busy and when the next time we would be available is. This allowed us to plan the deliverable better and achieve internal deadlines.

3. GROUP Q

4. I would like to improve my knowledge of stakeholder understanding. I think that with stakeholder understanding it is important as they are the target audience for the application and their needs are to be met even if it differs from our own since they are the target. My approach for that will consist of understanding interviews as well as conversations with the stakeholder during testing. Another approach will be different user groups to see if our initial persona and ideas for the stakeholder would make sense.

I think this is important since it is easy to develop requirements by making assumptions but to bridge the gap to the target audience is crucial to make a product or service that they would actually use.

Cieran - Reflection

1. Team communication as always but I won't expand further on that. There was a decision to split the team into two subteams; a proof of concept team to begin the implementation and a verification and validation team to tackle this document. I feel the subteams have both been incredibly efficient in their work and the communication between the two has not faltered. The proof of concept team worked well together in overall and in subteam meetings to grasp an understanding of the software packages we will be utilizing while the verification and validation team ensured the entire system was covered within the tests outlined in this document.
2. I expanded on it during the last deliverable, that each person had this 'own version' of the system in their mind and we were all building towards a different thing when the assignment started. That wasn't entirely gone here and we hit a few bumps worrying about what each of the others was including within the system while thinking we don't need

that to be included. Through team meetings and a general walkthrough of the flow of the program, we all got on the same page and managed to generate tests and feel as if we've properly outlined a system to ensure the system is designed properly.

3. GROUP Q

4. I feel my personal interaction with test suite development lacks for the requirements of this project. To advance this I will attempt to do the following:
 - (a) Follow online guides, walkthroughs, and documentation on the software packages (Python Extension, pytest, pylint, etc) to ensure the software testing suite developed contains both meaningful tests derived from this document while also covering all the software.
 - (b) Generate and execute tests in advancing complexity to ensure an understanding of the routines gone through with a testing suite. 'Getting my hands dirty'.

Group - Reflection

1. SOLO Q

2. SOLO Q

3. See below:

- System testing, unit testing: understanding the distinction and the overlap between them
- Database security: What the security vulnerabilities are for databases. Some network testing is also contained in this category. Tools such as Wireshark (network traffic analyzer) and Nmap (which machines are active on the network). Note: Wireshark usage has legal implications. The team will ensure that all testing is properly done.
- API testing (OpenCV): tool has not been selected yet.
- Negative and edge case testing: using Python scripts.

- How to integrate previously designed requirements into tests (trying to design tests that satisfy multiple requirements).
- Improve knowledge of OpenCV and MediaPipe pose detection for accurate testing of body landmark tracking.
- Understand how to verify angle accuracy and other issues during testing with OpenCV and MediaPipe.
- General knowledge of subject matter (physiotherapy)/stakeholder understanding (This involves an understanding of what should be tested and areas for success).
- Understanding what the movement is, which ones can be assessed and requirements for each exercise warrants.

4. **SOLO Q** (Based on GROUP Q above.)